



[Home](#) / [Manuals](#) / [Docker Compose](#) / [How-tos](#) / [Use Compose in production](#)

Use Compose in production

Table of contents

[Modify your Compose file for production](#)

[Deploying changes](#)

[Running Compose on a single server](#)

[Next steps](#)

When you define your app with Compose in development, you can use this definition to run your application in different environments such as CI, staging, and production.

The easiest way to deploy an application is to run it on a single server, similar to how you would run your development environment. If you want to scale up your application, you can run Compose apps on a Swarm cluster.

Modify your Compose file for production

You may need to make changes to your app configuration to make it ready for production. These changes might include:

- Removing any volume bindings for application code, so that code stays inside the container and can't be changed from outside
- Binding to different ports on the host
- Setting environment variables differently, such as reducing the verbosity of logging, or to specify settings for external services such as an email server
- Specifying a restart policy like `restart: always` to avoid downtime

- Adding extra services such as a log aggregator

For this reason, consider defining an additional Compose file, for example

`compose.production.yaml`, with production-specific configuration details. This configuration file only needs to include the changes you want to make from the original Compose file. The additional Compose file is then applied over the original `compose.yaml` to create a new configuration.

Once you have a second configuration file, you can use it with the `-f` option:

```
$ docker compose -f compose.yaml -f compose.production.yaml up -d
```

See [Using multiple compose files](#) for a more complete example, and other options.

Deploying changes

When you make changes to your app code, remember to rebuild your image and recreate your app's containers. To redeploy a service called `web`, use:

```
$ docker compose build web
$ docker compose up --no-deps -d web
```

This first command rebuilds the image for `web` and then stops, destroys, and recreates just the `web` service. The `--no-deps` flag prevents Compose from also recreating any services that `web` depends on.

Running Compose on a single server

You can use Compose to deploy an app to a remote Docker host by setting the `DOCKER_HOST`, `DOCKER_TLS_VERIFY`, and `DOCKER_CERT_PATH` environment variables appropriately. For more information, see [pre-defined environment variables](#).

Once you've set up your environment variables, all the normal `docker compose` commands work with no further configuration.

Next steps

- [Familiarize yourself with Compose's trust model](#)

- [Using multiple Compose files](#)